

S.Y.N.

Synthetic Yielding Nexus

Technical Learning Guide

Days 1 - 3: Foundation, Clap Detection, Speech-to-Text

Audio Fundamentals • Signal Processing • Neural Networks • Transformers

Generated: May 2026

S.Y.N. — Learning Guide: Days 1–3

*Everything you need to understand what you've built — from the ground up.
Read this, and you won't just know what the code does — you'll know why.*

Table of Contents

- [Day 1 — Foundation](#)
 - [1.1 Python Project Structure](#)
 - [1.2 How Text-to-Speech Works](#)
 - [1.3 The `\_\_name\_\_` Guard](#)
 - [Day 2 — Clap Detection](#)
 - [2.1 Digital Audio Fundamentals](#)
 - [2.2 Audio Streams & Buffers](#)
 - [2.3 RMS Energy — Measuring Loudness](#)
 - [2.4 Signal Detection & Thresholds](#)
 - [2.5 Pattern Recognition \(Double-Clap\)](#)
 - [2.6 Threading & Callbacks](#)
 - [2.7 Configuration Management](#)
 - [Day 3 — Speech-to-Text](#)
 - [3.1 Audio Recording & WAV Format](#)
 - [3.2 State Machines](#)
 - [3.3 The Fourier Transform — Sound to Frequencies](#)
 - [3.4 Mel Spectrograms — How Machines "See" Sound](#)
 - [3.5 Neural Networks 101](#)
 - [3.6 Transformers — The Architecture Behind Whisper](#)
 - [3.7 How Whisper Works End-to-End](#)
 - [3.8 Model Quantization — Speed vs Accuracy](#)
 - [3.9 Design Patterns Used](#)
 - [3.10 Thread Safety & Concurrency](#)
-

Day 1 — Foundation

1.1 Python Project Structure

Why structure matters

When you write a small script, one file is fine. But SYN will have 30+ files — without structure, you'd never find anything. Python uses **packages** (folders with `__init__.py`) to organize code.

The `__init__.py` file

```
backend/ __init__.py ← Makes "backend" a Python package voice/ __init__.py ← Makes "voice" a sub-package tts.py clap_detector.py
```

What it does: When Python sees a folder with `__init__.py`, it treats that folder as an importable package. Without it, `from backend.voice.tts import speak` would fail.

What goes inside it? Usually nothing (or just a comment). It's just a marker file. Advanced usage: you can put shared imports here so users can do `from backend.voice import speak` directly.

How imports work

```
# Absolute import — full path from project root from backend.voice.tts import speak #  
Relative import — relative to current file (used inside packages) from .tts import  
# same folder from . import # parent folder's sibling
```

[!TIP]

Always use **absolute imports** in SYN. They're clearer and don't break when you move files around.

How Python finds your modules

When you run `python main.py` from `C:\Users\feate\Desktop\SYN\`, Python adds that directory to its **search path** (`sys.path`). So `backend.voice.tts` means:

```
C:\Users\feate\Desktop\SYN\ + backend\voice\tts.py (sys.path) import path
```

1.2 How Text-to-Speech Works

The TTS Pipeline

Text → Linguistic Analysis → Acoustic Model → Waveform → Speaker

Step 1 — Linguistic Analysis:

Break text into phonemes (basic sounds).

"Hello" → /h/ /e/ /l/ /o/ "sYN" → /s/ /i/ /n/

This step also handles:

- Abbreviations: "Dr." → "Doctor"
- Numbers: "42" → "forty two"
- Punctuation: periods = pause, question marks = rising pitch

Step 2 — Acoustic Model:

Convert phonemes into audio parameters (pitch, duration, speed).

This is where the "voice" character comes from.

Step 3 — Waveform Generation:

Generate actual audio samples (the wave your speaker plays).

pyttsx3 — Our TTS engine

pyttsx3 is an **offline** TTS library. It wraps your OS's built-in speech engine:

- **Windows:** Microsoft SAPI5 (the same voice as Cortana/Narrator)
- **macOS:** NSSpeechSynthesizer
- **Linux:** espeak

```
import pyttsx3 engine = pyttsx3.init() # connects to OS speech engine
engine.setProperty("rate", 175 # words per minute engine.setProperty("volume", 1.0
engine.say("Hello") # queues the text engine.runAndWait() # blocks until speech
finishes
```

[!NOTE]

Offline vs Cloud TTS: *pyttsx3 works without internet but sounds robotic. ElevenLabs (cloud) sounds human-like but needs internet + API key + costs money. We start offline, upgrade later.*

Key properties

Property	What it controls	Typical values
<code>rate</code>	Speed (words/min)	120 (slow) → 200 (fast)
<code>volume</code>	Loudness	0.0 (silent) → 1.0 (max)
<code>voice</code>	Which voice to use	Depends on OS installed voices

1.3 The `__name__` Guard

```
if __name__ == "__main__":
```

What this does:

- When you run `python main.py` directly → `__name__` is `"__main__"` → code runs
- When another file does `import main` → `__name__` is `"main"` → code does NOT run

Why it matters: Without this guard, importing `main.py` from another file would accidentally start the whole SYN boot sequence. The guard says: *"Only run this if I'm the main script, not if I'm being imported."*

Day 2 — Clap Detection

2.1 Digital Audio Fundamentals

Sound is a wave

Sound is vibrations in air — areas of high pressure and low pressure alternating rapidly. Your ear detects these pressure changes. A microphone does the same thing electronically.

Sampling — How computers capture sound

A computer can't store a continuous wave. Instead, it **samples** the wave thousands of times per second — like taking snapshots of the wave's position.

```
Continuous wave: ~~~~[wavy line]~~~~ Sampled (low rate): . . . . . Takes few snapshots -  
choppy, loses detail Sampled (high rate): . . . . . Many snapshots -  
smooth, accurate
```

Key audio terms

Term	What it means	Our setting	Why
Sample Rate	Snapshots per second	44,100 Hz	CD quality. Captures up to 22,050 Hz (all human hearing)

Bit Depth	Precision of each snapshot	16-bit	65,536 possible values per sample. Good enough for voice
Channels	Number of independent streams	1 (mono)	We don't need stereo for voice commands
Chunk Size	Samples per buffer read	1,024	~23ms of audio per chunk. Good balance of speed vs CPU

The Nyquist Theorem

*To capture a frequency, you need to sample at **2× that frequency**.*

Human hearing goes up to ~20,000 Hz. So we need at least 40,000 Hz sampling. That's why CD audio is 44,100 Hz — safely above $2 \times 20,000$.

For SYN: Claps have energy in the 1,500–8,000 Hz range. Our 44,100 Hz sample rate captures all of it easily.

How samples are stored

Each sample is a **signed 16-bit integer**: a number from -32,768 to +32,767.

- **0** = silence (no air pressure change)
- **+32,767** = maximum positive pressure
- **-32,768** = maximum negative pressure

```
Loud sound: [28000, -31000, 29500, -27000, ...] (big numbers) Quiet sound: [120, -85, 95, -110, ...] (small numbers) Silence: [3, -2, 1, -4, ...] (near zero)
```

2.2 Audio Streams & Buffers

What is a stream?

Instead of recording all audio then processing it, we read audio in a continuous **stream** — chunk by chunk, in real-time. Think of it like drinking from a water hose vs. filling a bucket first.

PyAudio — The bridge to your microphone

```
import pyaudio
pa = pyaudio.PyAudio() # Open a stream (starts reading from mic)
stream = pa.open(format=pyaudio.pa_int16, channels=1, sample_rate=44100, # 16-bit samples =1 # mono =44100 #
```

```
44.1 kHz input=True # we're READING (not playing) frames per buffer=1024 # chunk size
# Read one chunk (1024 samples = ~23ms of audio) raw_bytes = stream.read(1024) #
raw_bytes is a bytes object: b'\x03\x00\xf8\xff...' # Convert to numpy array for math:
import numpy as np num_samples = np.frombuffer(raw_bytes, dtype=np.int16) # samples is now:
array([3, -8, 120, -95, ...])
```

Why numpy?

Raw audio is just bytes. To do math (calculate loudness, detect spikes), we need **numbers**. NumPy converts bytes → integer arrays and does math **blazingly fast** (it's written in C under the hood).

Buffer overflow

The mic is always producing audio. If your code is too slow to read chunks, the buffer fills up and overflows → you lose audio. That's why we use `exception_on_overflow=False` — it silently drops old data instead of crashing.

2.3 RMS Energy — Measuring Loudness

The problem

How do you measure "how loud" a chunk of audio is? You can't just average the samples, because audio oscillates between positive and negative:

```
Samples: [500, -480, 510, -490, 500, -510] Average: (500 + -480 + 510 + -490 + 500 +
-510) / 6 = 5 ← almost zero!
```

The average is nearly zero even though the sound is quite loud!

The solution — RMS (Root Mean Square)

```
Step 1: SQUARE every sample (makes everything positive) [500, -480, 510, -490, 500,
-510] → [250000, 230400, 260100, 240100, 250000, 260100] Step 2: MEAN (average the
squared values) (250000 + 230400 + 260100 + 240100 + 250000 + 260100) / 6 = 248,450
Step 3: ROOT (square root to get back to original scale) √248,450 ≈ 498.4
```

Result: RMS = 498.4 — correctly reflects the loudness!

In code

```
rms = float(np.sqrt(np.mean(audio_chunk.astype(np.int16) ** 2)))
```

Breaking it down:

```
audio_chunk.astype(np.float64) # convert int16 → float64 (avoids overflow) ** 2 #
square every sample np.mean(...) # average the squares np.sqrt(...) # square root
float(...) # convert numpy scalar → Python float
```

[!IMPORTANT]

Why `.astype(np.float64)`? `int16` values go up to 32,767. Squaring that gives 1,073,741,824 — which overflows `int16` (max 32,767) and even `int32` (max ~2 billion). `float64` handles it safely.

What RMS values mean in practice

RMS Range	What it sounds like	SYN context
0–100	Silence / very quiet room	Background noise
100–400	Quiet talking, ambient noise	Below our threshold
400–800	Normal speech, keyboard typing	Speech detection
800–3000	Loud clap, shout, door slam	Clap detection!
3000+	Very loud sound	Definitely a clap

2.4 Signal Detection & Thresholds

Simple threshold detection

The most basic approach: if $\text{RMS} > \text{threshold}$, something happened.

```
Energy over time: ____ | | | | 900 | 850 | ← above threshold (1000) ____
|____| |____| ← threshold line at 1000 time → Problem: speech, music, and dropped
objects also cross the threshold!
```

Spike duration — what makes a clap special?

A clap is unique because it's **very short** (5–80 milliseconds). Speech is longer (200ms+). Music is continuous.

```
Clap: ____|█|____ (sharp spike, ~30ms) Speech: ____|██████|____ (sustained energy, ~500ms)
Music: ____|██████████|____ (continuous energy)
```

Our detector checks:

```
if self.min_duration ms <= spike duration ms <= self.max_duration ms: # 5ms ≤ duration ≤ 80ms → it's a clap!
```

This one check eliminates most false positives.

Calibration — adapting to your environment

Every room has different background noise (fan, AC, traffic). The `calibrate_clap_threshold()` function:

1. Listens for 5 seconds of "silence"
2. Measures the peak background noise
3. Sets threshold to 3× that peak

Why 3×? A clap should be significantly louder than any background noise. 3× gives a safe margin — loud enough that only real claps trigger it, low enough that you don't have to clap absurdly hard.

2.5 Pattern Recognition (Double-Clap)

Why double-clap instead of single?

Single clap has too many false positives — a book falling, a door closing, or a loud footstep could trigger it. Double-clap is much more intentional.

The timing window

```
Clap 1 Clap 2 | | | ← 0.8s → | | window | If Clap 2 arrives within 0.8s of Clap 1  
→ TRIGGER! If Clap 2 arrives after 0.8s → reset, treat as new Clap 1
```

State flow

```
IDLE → [Clap detected] → WAITING FOR SECOND → [second clap within 0.8s] → TRIGGERED!  
→ [clap after 0.8s] → IDLE → [clap after 0.8s] → IDLE
```

Cooldown

After triggering, we ignore all claps for 2 seconds. Why?

- SYN speaks after waking up ("I'm listening")
- The speaker sound could be picked up by the mic
- Without cooldown, SYN could trigger itself in a loop!

2.6 Threading & Callbacks

What is a thread?

A thread is an independent path of execution. Your program normally runs one thing at a time (single-threaded). With threads, you can do multiple things "simultaneously."

```
single-threaded: [clap detection] → [blocked: speaking] → [clap detection] → ...
Problem: Can't detect claps while speaking! Multi-threaded: thread 1: [clap detection]
[blocked: speaking] thread 2: [clap detection] [blocked: speaking] Each runs at the
same time!
```

Daemon threads

```
thread = threading.Thread(target=on_wake, daemon=True)
```

`daemon=True` means: "If the main program exits, kill this thread too."

Without it, background threads can keep your program alive even after Ctrl+C.

Callbacks — the "don't call me, I'll call you" pattern

Instead of the clap detector returning a value, it calls a function YOU provide:

```
def my_handler(): print("Woke up!") detector = ClapDetector(on_wake=my_handler) # "call
my_handler when you detect a clap" detector.start()
```

Why callbacks? Because the clap detector runs in a loop — it can't "return" a clap to you. Instead, it calls your function when the event happens. This is called **event-driven programming** — the same pattern used in web apps (onClick), game engines (onCollision), and GUIs (onButtonPress).

2.7 Configuration Management

Why a central config file?

Without it, magic numbers are scattered everywhere:

```
# Bad: numbers buried in code, no context if energy > 2500 # what is 2500? why 2500?
time.sleep(0.8) # what is 0.8?
```

With a config file:

```
# Good: named, documented, easy to change if energy > config.CLAP_ENERGY_THRESHOLD
time.sleep(config.CLAP_DOUBLE_TAP_WINDOW)
```

Benefits

1. **Single source of truth** — change a value in one place, affects entire project

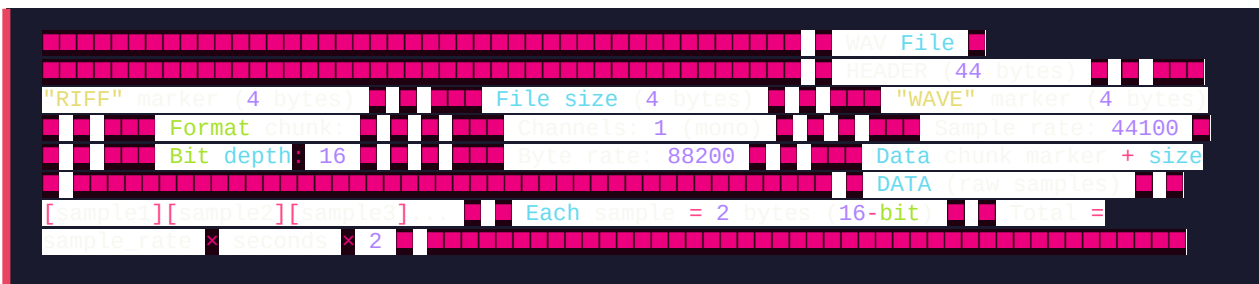
2. **Self-documenting** — config names explain what values mean
3. **Easy tuning** — adjust without digging through code
4. **Environment-specific** — different settings for different rooms/mics

Day 3 — Speech-to-Text

3.1 Audio Recording & WAV Format

WAV file structure

WAV is the simplest audio format. No compression, no codec — just raw numbers with a header.



Example: 3 seconds of mono audio at 44,100 Hz, 16-bit:

```
Header: 44 bytes Data: 44,100 × 3 × 2 = 264,600 bytes Total: 264,644 bytes = 258 Kb
```

Why WAV and not MP3?

- WAV is **lossless** — no information is thrown away
- Whisper expects raw audio data
- No decoding step needed (faster)
- WAV is trivial to create with Python's **wave** module
- We delete the file after transcription anyway, so size doesn't matter

Python's **wave** module

```
import wave with wave.open("output.wav", "wb") as wf: wf.setchannels(1) # mono
wf.setsampwidth(2) # 2 bytes = 16-bit wf.setframerate(44100) # sample rate
wf.writeframes(raw_bytes) # the actual audio data
```

[!NOTE]

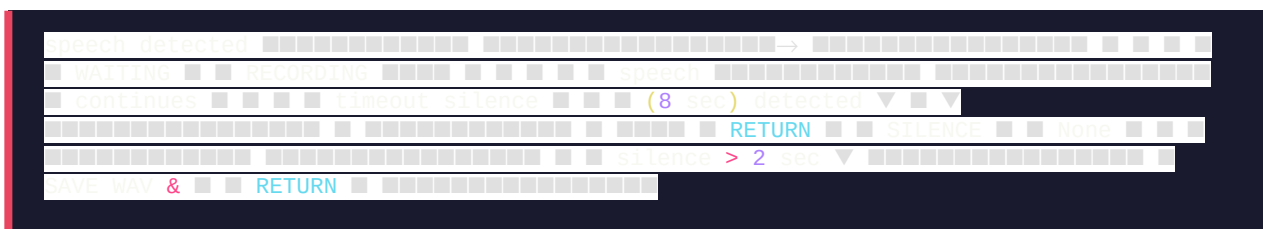
Temporary files: We use `tempfile.gettempdir()` to store recordings. This is your OS's temp folder (`C:\Users\...\AppData\Local\Temp`). Files here are cleaned up automatically by the OS eventually, but we also delete them ourselves right after transcription.

3.2 State Machines

What is a state machine?

A system that can be in one of several **states**, and transitions between them based on **events**.

SYN's audio recorder — a 3-state machine



Where state machines appear in SYN

Component	States	Transitions
Audio recorder	WAITING → RECORDING → SILENCE → DONE	Speech/silence energy
Clap detector	IDLE → FIRST_CLAP → TRIGGERED → COOLDOWN	Clap events + timing
SYN itself (future)	SLEEPING → LISTENING → PROCESSING → RESPONDING	Clap, speech, intent

Why state machines matter

They're one of the most fundamental patterns in programming:

- **Game characters:** IDLE → WALKING → ATTACKING → DEAD
- **Network connections:** DISCONNECTED → CONNECTING → CONNECTED → CLOSING
- **UI buttons:** DEFAULT → HOVER → PRESSED → DISABLED
- **Traffic lights:** GREEN → YELLOW → RED → GREEN

Once you can spot the states and transitions, you can model almost any system.

3.3 The Fourier Transform — Sound to Frequencies

The core idea

Any sound — music, speech, claps — is actually a **mix of simple sine waves** at different frequencies. The Fourier Transform separates them.

```
Mixed sound wave: [complex shape] Fourier Transform ▼ Separated: Low: [bass: 200 Hz] Mid: [voice: 1000 Hz] High: [treble: 5000 Hz]
```

Analogy — a prism for sound

A prism splits white light into individual colors (frequencies).
The Fourier Transform splits complex sound into individual frequencies.

```
White light → [PRISM] → Red, Orange, Yellow, Green, Blue, Violet  
Complex sound → [FFT] → 100 Hz, 200 Hz, 500 Hz, 1 kHz, 2 kHz, 5 kHz, ...
```

Why it matters for speech recognition

Raw audio is just a wiggly line — hard for an AI to understand. But a frequency breakdown reveals **structure**:

- Vowels have strong low frequencies (200–1000 Hz)
- Consonants like "s" and "t" have high frequencies (3000–8000 Hz)
- Each phoneme has a unique frequency "fingerprint"

FFT — the Fast Fourier Transform

The FFT is an efficient algorithm to compute the Fourier Transform. It takes N samples and outputs $N/2$ frequency bins:

```
import numpy as np  
fft_result = np.fft.fft(audio_chunk) # complex numbers  
np.abs(fft_result) # how loud each frequency is  
np.fft.fftfreq(len(audio_chunk), 1/44100) # which frequencies
```

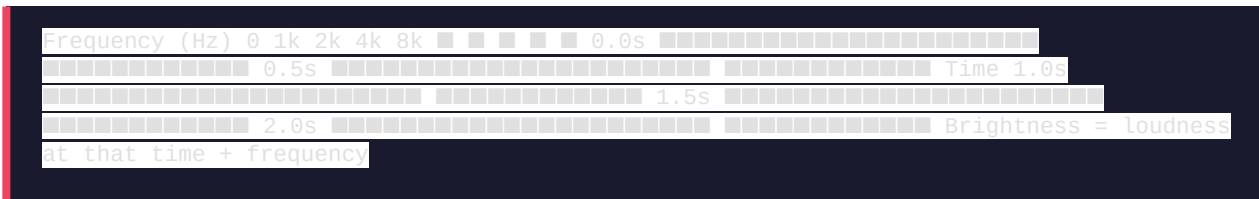
[!TIP]

You don't need to understand the math deeply to use Whisper — it handles all of this internally. But understanding FFT helps you grasp why spectrograms exist and how audio AI works.

3.4 Mel Spectrograms — How Machines "See" Sound

From 1D wave to 2D image

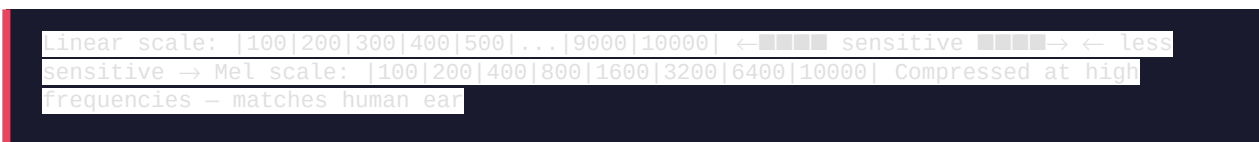
A spectrogram applies FFT to small overlapping windows of audio, creating a 2D representation:



- **X-axis:** Time (left to right)
- **Y-axis:** Frequency (low to high)
- **Brightness:** How loud that frequency is at that moment

Why "Mel" scale?

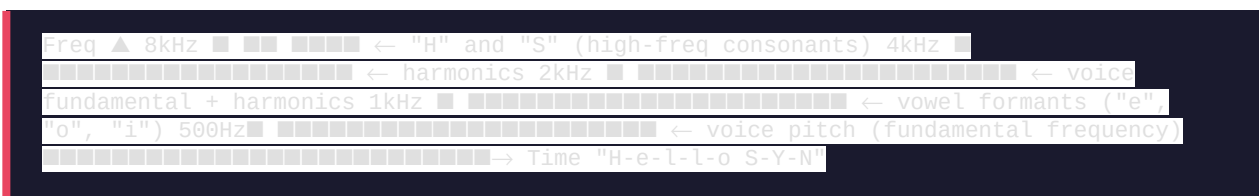
Human hearing is **not linear**. We can easily tell the difference between 200 Hz and 400 Hz, but 10,000 Hz and 10,200 Hz sound almost the same.



The Mel scale warps frequencies to match human perception. This gives Whisper better features to work with — it focuses detail where it matters.

What Whisper sees

When you say "Hello SYN", Whisper sees something like:



3.5 Neural Networks 101

What is a neural network?

A function that learns patterns from data. It takes an input, passes it through layers of "neurons," and produces an output.



Each line has a "weight" (a number). Training adjusts these weights to minimize errors.

Key terms

Term	Meaning	Analogy
Neuron	A unit that takes inputs, multiplies by weights, adds bias, applies activation	A single decision-maker
Layer	A group of neurons at the same level	A department in a company
Weights	Numbers that determine how important each input is	How much you trust each source
Training	Adjusting weights to reduce errors on known data	Studying for an exam
Inference	Using trained weights to process new data	Taking the exam
Parameters	Total count of weights in the model	Whisper base has 74 million

Why deep learning works

Each layer learns increasingly abstract features:

Layer 1: Edges, basic shapes (low-level audio patterns) Layer 2: Phonemes, syllables ("eh", "lo", "sin") Layer 3: Words, phrases ("Hello", "SYN") Layer 4: Context, meaning (understanding the sentence)

3.6 Transformers — The Architecture Behind Whisper

The revolution

Before Transformers (2017), AI processed sequences one step at a time (slow). Transformers process **all steps simultaneously** using a mechanism called **attention**.

Self-Attention — the key insight

"To understand a word, look at all the other words and decide which ones are important."

"I need to open the Chrome browser" when processing "Chrome": "I" → low attention (not relevant) "need" → low attention "open" → HIGH attention (action related to Chrome) "the" → low attention "Chrome" → self (always high) "browser" → HIGH attention (describes Chrome)

The model learns which words to pay attention to through training.

Encoder-Decoder Architecture (Whisper's design)



Encoder: Reads the entire mel spectrogram and creates a rich representation of the audio (what sounds are where).

Decoder: Generates text one token at a time, using attention to look at the encoder's audio representation. "Given what I've heard, the next word is probably..."

3.7 How Whisper Works End-to-End

The complete pipeline

Step 1 LOAD AUDIO Read WAV file → get raw samples Resample to 16 000 Hz Whisper's expected input rate Pad or trim to 30 seconds Whisper processes fixed-length chunks

Step 2 MEL SPECTROGRAM Apply FFT with 25ms windows 10ms hop Convert to 80 Mel frequency bins Apply log scaling human perception is logarithmic Result 80 × 3000 matrix 80 freq bins × 3000 time steps

Step 3 ENCODER Passes the mel spectrogram through 6 transformer layers Each layer Self-Attention → Feed-Forward → Normalization Output 1500 × 512 hidden states compressed audio representation

Step 4 DECODER autoregressive - one token at a time Start with |startoftranscript| token Look at encoder output cross-attention to audio Predict next token "Hello" Feed "Hello" back in predict next "SYN" Continue until |endoftext| token

Step 5 POST-PROCESSING Join tokens into text Apply language-specific normalization Return timestamped segments

Model sizes and tradeoffs

Model	Parameters	RAM	Speed (CPU)	Accuracy	Best for
<code>tiny</code>	39M	~1 GB	Very fast	Decent	Quick testing

base	74M	~1 GB	Fast	Good	Daily use on CPU ← we use this
small	244M	~2 GB	Medium	Very good	Better accuracy when needed
medium	769M	~5 GB	Slow	Excellent	Professional use
large-v3	1.5B	~10 GB	Very slow	Best	Maximum accuracy, needs GPU

faster-whisper vs original Whisper

Feature	OpenAI Whisper	faster-whisper
Framework	PyTorch	CTranslate2
CPU speed	1× (baseline)	4× faster
RAM usage	Higher	~50% less
Accuracy	Baseline	Same (same model weights)
GPU support	CUDA	CUDA + INT8 quantization

[!TIP]

CTranslate2 is an inference optimization engine. It takes a trained model and makes it run faster by: fusing operations, using optimized CPU instructions (AVX, SSE), and quantizing weights. Same accuracy, less compute.

3.8 Model Quantization — Speed vs Accuracy

What is quantization?

Reducing the precision of model weights to make inference faster.

Why it works

Our setting

compute_type	Precision	Speed	Accuracy loss	When to use
float32	Full	1× (slowest)	None	When accuracy is critical
float16	Half	2× faster	Negligible	GPU (CUDA) only
int8	Quarter	4× faster	Minimal	CPU — what we use

3.9 Design Patterns Used

Pattern 2: Lazy Loading (Whisper Model)

Problem: Loading the Whisper model takes ~3 seconds and uses ~1 GB RAM. We don't want to pay this cost at boot.

Solution: Don't load the model until someone actually calls `transcribe()` for the first time.

```
def _load_model(self): if self._model is not None: return # already loaded - skip
self._model = WhisperModel(...) # expensive - only runs once
def transcribe(self, wav_path): self._load_model() # loads on first call, skips on subsequent calls # ...
use self._model ...
```

Real-world examples of lazy loading:

- Web browsers load images as you scroll down (not all at once)
- Games load level assets when you enter the level
- Python imports modules when you first `import` them

Pattern 3: Event Loop

Problem: SYN needs to wait for events (claps) indefinitely, handle them, and go back to waiting.

Solution: An infinite loop that sleeps until an event, processes it, and repeats.

```
wait for event (clap detection)
event!
handle event (record + syn)
result (speak response)
```

Used everywhere:

- Web servers: wait for request → process → respond → repeat
- GUIs: wait for click → handle → update UI → repeat
- Game engines: wait for frame → input → update → render → repeat

3.10 Thread Safety & Concurrency

The problem: race conditions

When two threads access the same variable simultaneously, unpredictable things happen:

```
Thread 1 (clap detector): is_processing = True Thread 2 (another clap): reads
is_processing → still False (stale!) starts processing too → CRASH/DOUBLE RESPONSE
```

The solution: locks

A **lock** (mutex) ensures only one thread can enter a critical section at a time:

```
self.lock = threading.Lock() def _on_wake(self): with self.lock: # Thread 1 enters,
Thread 2 WAITS here if self.is_processing: return # safely check without race
condition self.is_processing = True # ... do work ... self.is_processing = False #
release for next wake event
```



Key threading concepts

Concept	What it means	SYN usage
Thread	Independent execution path	Clap callback runs in a separate thread
Daemon thread	Dies when main program exits	Wake handler threads are daemons
Lock (Mutex)	Only one thread can hold it at a time	Prevents double wake processing
Race condition	Two threads modifying same data simultaneously	What locks prevent
Deadlock	Two threads waiting for each other's locks forever	Avoided by simple lock structure
Thread-safe	Code that works correctly with multiple threads	Our <code>ListenLoop._on_wake()</code>

[!WARNING]

Common mistake: Forgetting to release a lock after an exception. That's why we use `with self._lock:` — Python's context manager automatically releases the lock even if an error occurs inside the block.

Quick Reference — All Concepts Map

